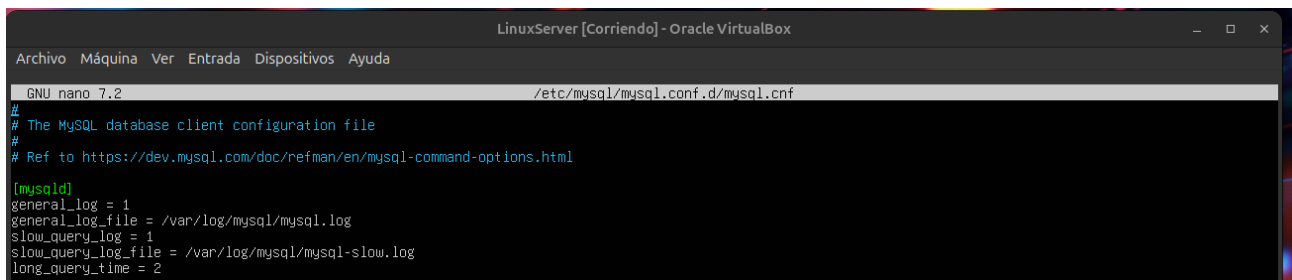


Reto: Bloque 4 día 29

Fase: 1

Revisaremos los logs de consultas y accesos en MySQL para detectar anomalías. Identificaremos usuarios con privilegios excesivos y accesos remotos no controlados. Veremos si existen debilidades SQL usando SQLmap. Y comprobaremos si la BD están cifradas y si el tráfico entre cliente y servidor usa TLS.

Partiremos de la base de que ya tenemos instalado MySQL. Para activar los logs en MySQL editaremos el archivo `mysql.cnf`. Usaremos el comando `sudo nano`.

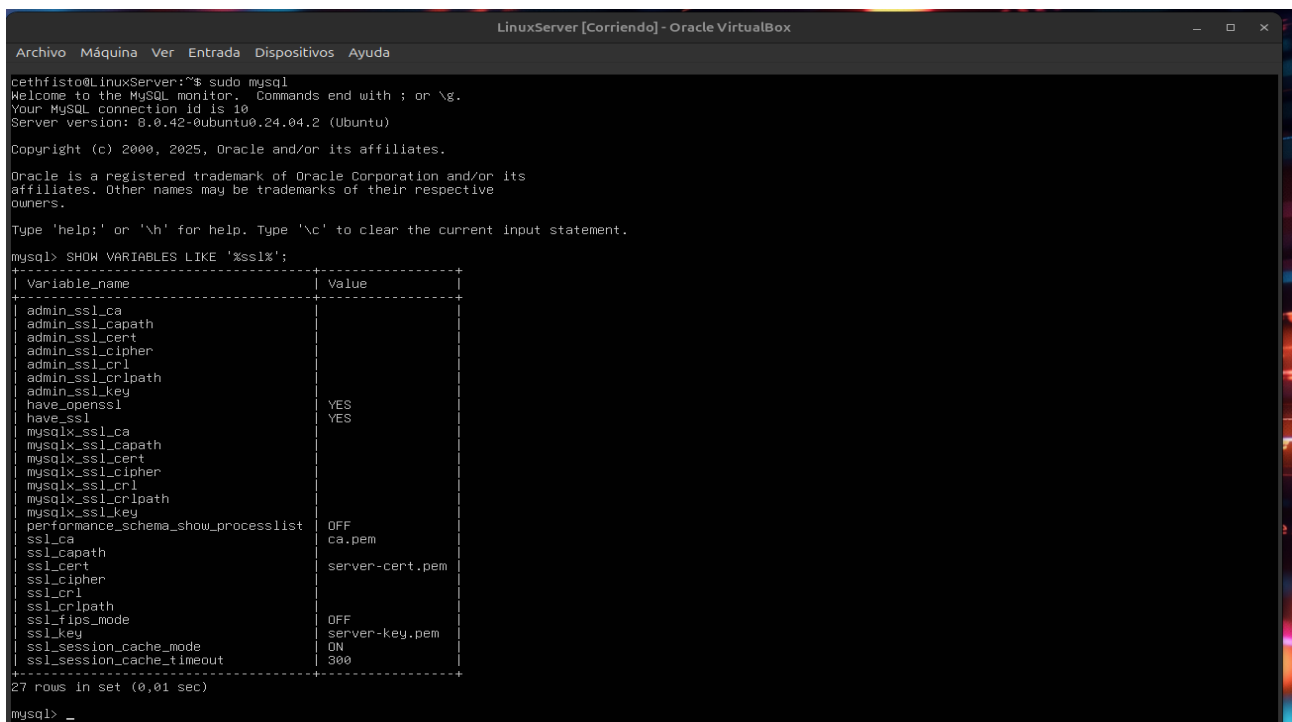


```
LinuxServer [Corriendo] - Oracle VirtualBox
Archivo Máquina Ver Entrada Dispositivos Ayuda
GNU nano 7.2 /etc/mysql/mysql.conf.d/mysql.cnf
#
# The MySQL database client configuration file
#
# Ref to https://dev.mysql.com/doc/refman/en/mysql-command-options.html
[mysqld]
general_log = 1
general_log_file = /var/log/mysql/mysql.log
slow_query_log = 1
slow_query_log_file = /var/log/mysql/mysql-slow.log
long_query_time = 2
```

* Esta sería una configuración para activar los logs en MySQL. Acto seguido reiniciamos el servicio.

Para ir revisando los logs creado en MySQL usaremos el comando `sudo tail -f /var/log/mysql/mysql.log`.

Para identificar a usuarios con demasiados privilegios. Usaremos el siguiente comando una vez iniciado MySQL. `SELECT user, host, super_priv FROM mysql.user;`



```
LinuxServer [Corriendo] - Oracle VirtualBox
Archivo Máquina Ver Entrada Dispositivos Ayuda
cethfisto@LinuxServer:~$ sudo mysql
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 10
Server version: 8.0.42-0ubuntu0.24.04.2 (Ubuntu)

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> SHOW VARIABLES LIKE '%ssl%';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| admin_ssl_ca  |      |
| admin_ssl_capath |      |
| admin_ssl_cert |      |
| admin_ssl_cipher |      |
| admin_ssl_crl |      |
| admin_ssl_crlpath |      |
| admin_ssl_key |      |
| have_openssl  | YES  |
| have_ssl     | YES  |
| mysqlx_ssl_ca |      |
| mysqlx_ssl_capath |      |
| mysqlx_ssl_cert |      |
| mysqlx_ssl_cipher |      |
| mysqlx_ssl_crl |      |
| mysqlx_ssl_crlpath |      |
| mysqlx_ssl_key |      |
| performance_schema_show_processlist | OFF |
| ssl_ca       | ca.pem |
| ssl_capath   |      |
| ssl_cert     | server-cert.pem |
| ssl_cipher   |      |
| ssl_crl      |      |
| ssl_crlpath  |      |
| ssl_fips_mode | OFF  |
| ssl_key      | server-key.pem |
| ssl_session_cache_mode | ON  |
| ssl_session_cache_timeout | 300 |
+-----+-----+
27 rows in set (0.01 sec)

mysql> _
```

Para analizar accesos remotos no controlados, verificaremos que los usuarios no tengan el valor `host=%` o conexiones abiertas desde cualquier IP.

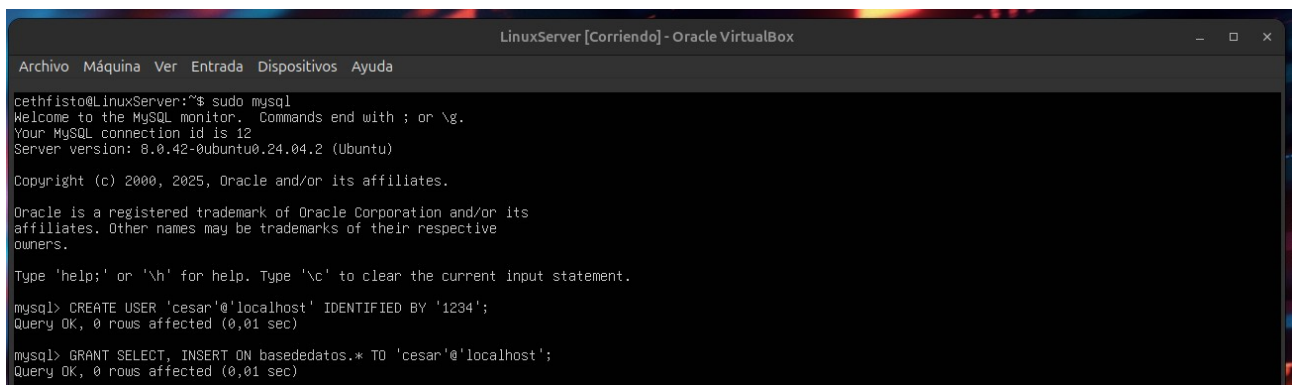
Para buscar debilidades con SQLmap usaremos el comando `sqlmap -u "URL de nuestra web" --batch --risk=3 --level=5`, que es un comando de SQLmap para saber si la URL introducida podría tener una inyección SQL.

Y para ver el cifrado de datos y conexiones usaremos el siguiente comando en MySQL, `SHOW VARIABLES LIKE '%ssl%'`;

Fase: 2

Configuraremos los roles y permisos de los usuarios. Activaremos el cifrado de datos en reposo y en tránsito. Restringiremos el acceso remoto solo a IPs autorizadas. Configuraremos el Firewall y las ACLs. E implementaremos los backups automatizados y cifrados para la recuperación de datos.

Para configurar usuarios con permisos mínimos en MySQL crearemos un usuario y le añadiremos los permisos.



```
LinuxServer [Corriendo] - Oracle VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda

cethfisto@LinuxServer:~$ sudo mysql
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 12
Server version: 8.0.42-0ubuntu0.24.04.2 (Ubuntu)

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

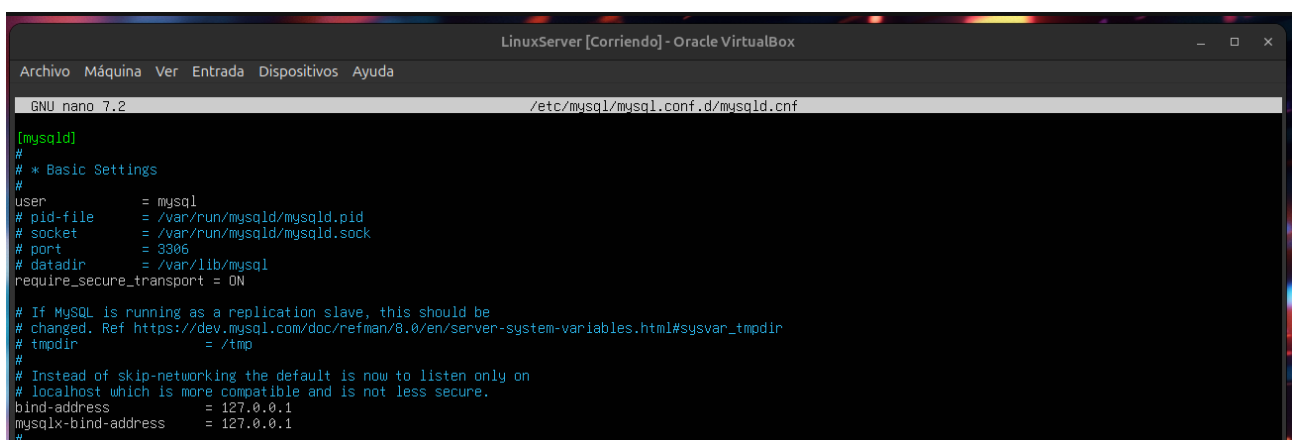
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE USER 'cesar'@'localhost' IDENTIFIED BY '1234';
Query OK, 0 rows affected (0.01 sec)

mysql> GRANT SELECT, INSERT ON basededatos.* TO 'cesar'@'localhost';
Query OK, 0 rows affected (0.01 sec)
```

* Aquí hemos creado un usuario llamado cesar y le hemos dado los permisos de select e insert en la BD llamada basededatos.

Para habilitar el cifrado SSL/TLS en MySQL debemos editar el archivo `mysqld.cnf`.



```
LinuxServer [Corriendo] - Oracle VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda

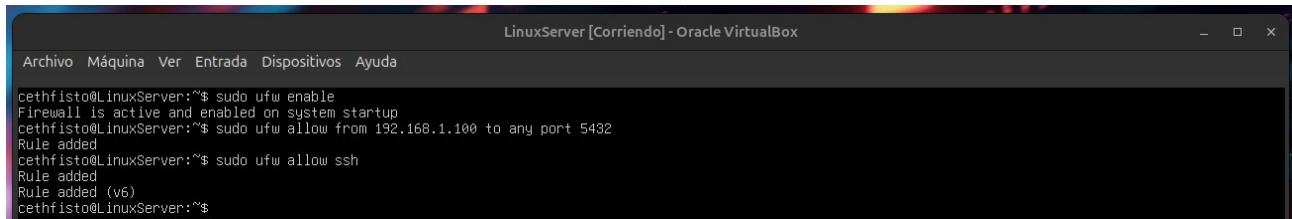
GNU nano 7.2 /etc/mysql/mysql.conf.d/mysqld.cnf

[mysqld]
#
# * Basic Settings
#
user                    = mysql
# pid-file              = /var/run/mysqld/mysqld.pid
# socket                = /var/run/mysqld/mysqld.sock
# port                  = 3306
# datadir               = /var/lib/mysql
require_secure_transport = ON

# If MySQL is running as a replication slave, this should be
# changed. Ref https://dev.mysql.com/doc/refman/8.0/en/server-system-variables.html#sysvar_tmpdir
# tmpdir                = /tmp
#
# Instead of skip-networking the default is now to listen only on
# localhost which is more compatible and is not less secure.
bind-address            = 127.0.0.1
mysqlx-bind-address     = 127.0.0.1
#
```

* También hemos aprovechado para añadir las IPs que estarán autorizadas.

Para configurar el firewall usaremos UFW.



```
LinuxServer [Corriendo] - Oracle VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda
cethfisto@LinuxServer:~$ sudo ufw enable
Firewall is active and enabled on system startup
cethfisto@LinuxServer:~$ sudo ufw allow from 192.168.1.100 to any port 5432
Rule added
cethfisto@LinuxServer:~$ sudo ufw allow ssh
Rule added
Rule added (v6)
cethfisto@LinuxServer:~$
```

* Aquí hemos activado el servicio y realizado una configuración de ejemplo.

Para automatizar los backups cifrados, deberemos crear un script y añadir el script a cron.

```
#!/bin/bash
```

```
# Variables
```

```
FECHA=$(date +"%Y-%m-%d_%H-%M")
```

```
DESTINO="/home/usuario/backups"
```

```
ARCHIVO="backup_$FECHA.sql"
```

```
ARCHIVO_CIFRADO="$ARCHIVO.gpg"
```

```
BASEDEDATOS="nombre_bd"
```

```
USUARIO="root"
```

```
CLAVE="tu_clave_mysql"
```

```
CLAVE_GPG="clave_super_secreta"
```

```
# Crear carpeta si no existe
```

```
mkdir -p "$DESTINO"
```

```
# Dump de la base de datos
```

```
mysqldump -u $USUARIO -p$CLAVE $BASEDEDATOS > "$DESTINO/$ARCHIVO"
```

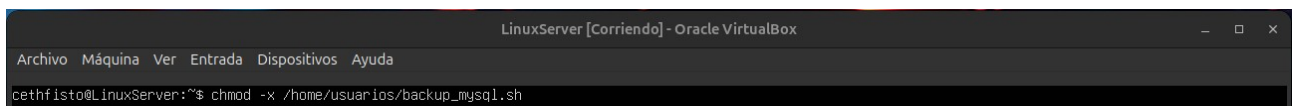
```
# Cifrar el archivo usando GPG simétrico
```

```
gpg --batch --yes --passphrase "$CLAVE_GPG" -c "$DESTINO/$ARCHIVO"
```

```
# Borrar archivo sin cifrar
```

```
rm "$DESTINO/$ARCHIVO"
```

Una vez creado el script asignamos los permisos de ejecución y lo añadimos al cron.



```
LinuxServer [Corriendo] - Oracle VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda
cethfisto@LinuxServer:~$ chmod -x /home/usuarios/backup_mysql.sh
```

```
LinuxServer [Corriendo] - Oracle VirtualBox
Archivo Máquina Ver Entrada Dispositivos Ayuda

# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow   command
0 4 * * * /home/usuarios/backup_mysql.sh
```

Fase: 3

Simularemos una inyección SQL controlada para verificar la seguridad del sistema. Simularemos un ataque de denegación del servicio enviando consultas pesadas a la BD. Y monitorizaremos los intentos de acceso no autorizados.

Para hacer una inyección controlado en MySQL, debemos crear un formulario web para la inyección y usar SQLmap para analizar la prueba.

Los pasos a realizar son:

1. Creamos el entorno de prueba instalando Apache, PHP y sus herramientas.
2. Crearemos la BD de prueba.
3. Creamos el formulario PHP para la prueba.
4. Usaremos SQLmap para detectar la inyección.

```
LinuxServer [Corriendo] - Oracle VirtualBox
Archivo Máquina Ver Entrada Dispositivos Ayuda

cethfisto@LinuxServer:~$ sudo mysql
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 33
Server version: 10.11.13-MariaDB-0ubuntu0.24.04.1 Ubuntu 24.04

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> CREATE DATABASE test;
ERROR 1007 (HY000): Can't create database 'test'; database exists
MariaDB [(none)]> USE test;
Database changed
MariaDB [test]> CREATE TABLE usuarios (
  -> id INT AUTO_INCREMENT PRIMARY KEY,
  -> nombre VARCHAR(50),
  -> clave VARCHAR(50)
  -> );
Query OK, 0 rows affected (0.014 sec)

MariaDB [test]> _
```

* Aquí hemos creado la BD de prueba.

Para simular un ataque DoS a la BD de prueba, crearemos un script que envíe muchas consultas pesadas.

```
LinuxServer [Corriendo] - Oracle VirtualBox
Archivo Máquina Ver Entrada Dispositivos Ayuda

GNU nano 7.2 test.sh
while true; do psql -U postgres -c "SELECT test(5);" & done
```

* Este podría ser un ejemplo de script para simular el ataque de DoS.

```
LinuxServer [Corriendo] - Oracle VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda

0[          0.0%] Tasks: 29, 35 thr, 82 kthr; 1 running
1[          0.0%] Load average: 0.02 0.07 0.07
Mem[|||||] 297M/3.82G Uptime: 01:24:01
Sup[      0K/0K]

Main  I/O
PID USER      PRI  NI  VIRT   RES   SHR  S  CPU% MEM%   TIME+  Command
14065 cethfisto   20    0 8684   4992  3712  R   0.7   0.1   0:00.02 http
1 root      20    0 22464 13412 9316  S   0.0   0.3   0:02.61 /sbin/init splash noprompt noshell automatic-ubiquity
301 root     19   -1 66764 17792 16640 S   0.0   0.4   0:00.28 /usr/lib/systemd/systemd-journald
351 root      RT    0 282M 27136 8704  S   0.0   0.7   0:00.08 /sbin/multipathd -d -s
364 root      RT    0 282M 27136 8704  S   0.0   0.7   0:00.00 /sbin/multipathd -d -s
366 root      RT    0 282M 27136 8704  S   0.0   0.7   0:00.00 /sbin/multipathd -d -s
367 root      RT    0 282M 27136 8704  S   0.0   0.7   0:00.00 /sbin/multipathd -d -s
368 root      RT    0 282M 27136 8704  S   0.0   0.7   0:00.00 /sbin/multipathd -d -s
```

* Luego usaremos el comando htop para ver cómo afectaría al rendimiento el ataque DoS.

Para detectar intentos de acceso no autorizados visualizaremos el archivo auth.log. Usaremos el comando `sudo cat /var/log/auth.log`. Podremos instala FAl2ban para bloquear accesos repetidos.

Fase: 4

Activaremos los logs de consultas y acceso con rotación de archivos de registro. Añadiremos alertas ante intentos de acceso fuera del horario laboral o ubicaciones desconocidas. Integraremos la BD con un IDS.

Para activar los logs avanzados, deberemos editar el archivo `mysql-server` y aplicar la configuración según las necesidades. Usaremos el comando `sudo nano /etc/logrotate.d/mysql-server`.

```
LinuxServer [Corriendo] - Oracle VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda

GNU nano 7.2 /etc/logrotate.d/mysql-server.dpkg-bak
# Flush all mysql server logs together using mysqladmin flush-logs.
# mysql must be active for the script to run successfully.
/var/log/mysql.log /var/log/mysql/*log {
    daily
    rotate 7
    missingok
    create 640 mysql adm
    su mysql adm
    compress
    sharedscripts
    postrotate
        # Confirm mysqladmin exists as it is needed for flushing logs.
        # If not, ignore this rotation as mysql-server has likely been removed.
        test -x /usr/bin/mysqladmin || exit 0
    endscript
}
```

Para crear alertas automatizadas de accesos sospechosos en MySQL, crearemos un script de monitorización, daremos los permisos de ejecución y lo añadimos al cron para que se ejecute cada 10 minutos.

```
#!/bin/bash
```

```
# Ruta del log de errores de MySQL
```

```
LOG="/var/log/mysql/error.log"
```

```
# Archivo donde se guarda la última línea leída
```

```
STATE="/tmp/mysql_last_line"
```

```
# Obtener número de línea anterior
```

```
LAST_LINE=$(cat $STATE 2>/dev/null || echo 0)
```

```
# Obtener número de líneas actuales
```

```
TOTAL_LINES=$(wc -l < $LOG)
```

```
# Extraer las líneas nuevas desde la última ejecución
```

```
if [ "$TOTAL_LINES" -gt "$LAST_LINE" ]; then
```

```
    TAIL_LINES=$((TOTAL_LINES - LAST_LINE))
```

```
    grep "Access denied" <(tail -n $TAIL_LINES $LOG) | grep -v "localhost" | while read line; do
```

```
        echo "🔒 ;Intento de acceso fallido detectado!: $line" | mail -s "Alerta de seguridad MySQL"
```

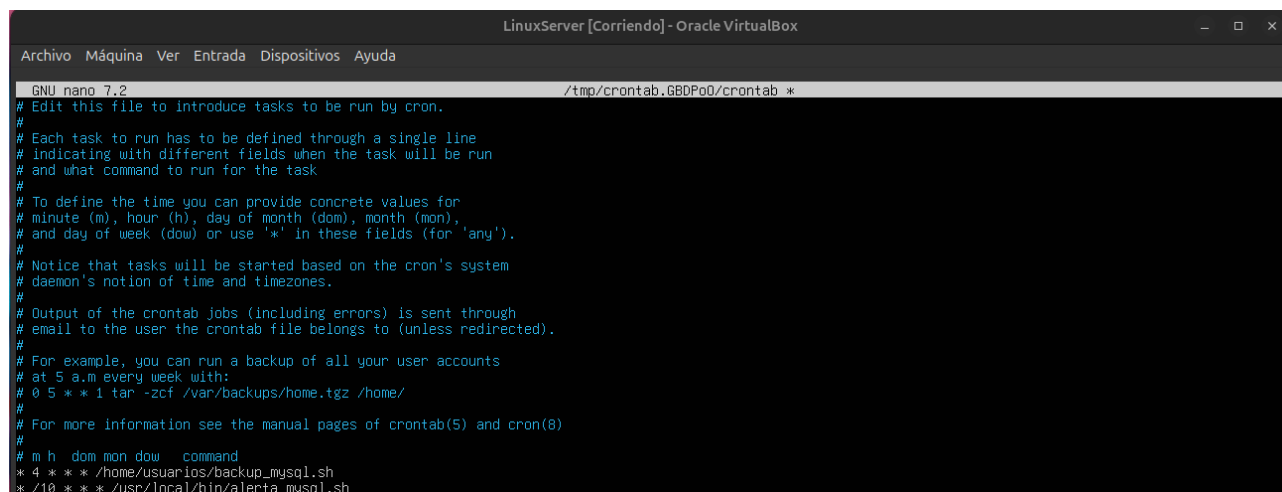
```
        tuemail@example.com
```

```
    done
```

```
    echo $TOTAL_LINES > $STATE
```

```
fi
```

* Este podría ser un ejemplo de script de monitorización. Ahora deberíamos añadirlo al cron.



```
LinuxServer [Corriendo] - Oracle VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda
GNU nano 7.2 /tmp/crontab.GBDPo0/crontab *
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow command
* 4 * * * /home/usuarios/backup_mysql.sh
*/10 * * * /usr/local/bin/alerta_mysql.sh
```

* Aquí hemos añadido el script de monitorización llamado alerta_mysql.sh al cron junto al cron de los backups.

Para integrar una BD con un IDS usaremos OSSEC.

Para ello deberemos seguir una serie de pasos:

1. Instalaremos OSSEC en el servidor usando el comando wget para descargar el repositorio de Github, los descomprimiremos y yendo al directorio de la descompresión, lo instalamos usando el comando sudo ./install.sh.

2. Configuraremos OSSEC para que analice los logs de MySQL.
3. Reiniciamos el servicio de OSSEC.

Podremos ver las alertas generadas por accesos sospechosos en el archivo alerts.log. Usaremos el comando `sudo nano /var/ossec/logs/alerts/alerts.log`.

Fase: 5

Al aplicar estas medidas ganaríamos una serie de fortalezas a la seguridad de la empresa de CodeArts para evitar la fuga de información.

Las mejoras que ganaríamos son:

- Análisis de debilidades.
- Implementación de medidas de seguridad, para reforzar el sistema.
- Pruebas de seguridad y simulación de ataques.
- Monitorización y auditoría de accesos y registros.